

Article

A Software Defect Prediction Method That Simultaneously Addresses Class Overlap and Noise Issues after Oversampling

Renliang Wang *, Feng Liu and Yanhui Bai

School of Computer Science and Technology, Beijing Jiaotong University, No. 3 Shangyuncun Haidian District, Beijing 100044, China; fliu@bjtu.edu.cn (F.L.); baiyh@bjtu.edu.cn (Y.B.)

* Correspondence: 21111099@bjtu.edu.cn

Abstract: Software defect prediction datasets often suffer from issues such as class imbalance, noise, and class overlap, making it difficult for classifiers to identify instances of defects. In response, researchers have proposed various techniques to mitigate the impact of these issues on classifier performance. Oversampling is a widely used method to address class imbalance. However, in addition to inherent noise and class overlap in the datasets themselves, oversampling methods can introduce new noise and class overlap while addressing class imbalance. To tackle these challenges, we propose a software defect prediction method called AS-KDENN, which simultaneously improves the effects of class imbalance, noise, and class overlap on classification models. AS-KDENN first performs oversampling using the Adaptive Synthetic Sampling Method (ADASYN), followed by our proposed KDENN method to address noise and class overlap. Unlike traditional methods, KDENN takes into account both the distance and local density information of overlapping samples, allowing for a more reasonable elimination of noise and instances of overlapping. To demonstrate the effectiveness of the AS-KDENN method, we conducted extensive experiments on 19 publicly available software defect prediction datasets. Compared to four commonly used oversampling techniques that also address class overlap or noise, the AS-KDENN method effectively alleviates issues of class imbalance, noise, and class overlap, subsequently improving the performance of the classifier models.



Citation: Wang, R.; Liu, F.; Bai, Y. A Software Defect Prediction Method That Simultaneously Addresses Class Overlap and Noise Issues after Oversampling. *Electronics* **2024**, *13*, 3976. <https://doi.org/10.3390/electronics13203976>

Academic Editors: Frédérique Ducroquet and George Angelos Papadopoulos

Received: 1 September 2024

Revised: 28 September 2024

Accepted: 8 October 2024

Published: 10 October 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: software defect prediction; class overlap; data quality; noise filtering; imbalanced learning

1. Introduction

Software Defect Prediction (SDP) technology can assist users in proactively identifying potential defect modules during the software development and testing phases. Compared to manual testing, software defect prediction technology can reduce testing costs and enable the timely localization and discovery of defect modules, thereby preventing significant losses for users due to software defects after deployment [1]. Therefore, software defect prediction has become a relatively active and important research area in recent years.

The software defect prediction process often faces issues such as class imbalance, class overlap, and noise [2–6]. In software defect prediction datasets, there are usually fewer non-defective modules than defective ones, leading to class imbalance between defective and non-defective module datasets [7–9]. Software defect prediction data primarily consist of feature values and labels of different category instances; when the feature values of these different instances share similarities, class overlap occurs [3]. Additionally, oversampling methods can also generate or exacerbate class overlap issues [10]. Noise typically arises from non-defective modules being incorrectly reported as defective [6], or due to defects being in a dormant state, causing defect modules to be mislabeled as non-defective [11,12]. Research by Pandey et al. [13] and Herzig et al. [6] has found that varying degrees of mislabeled data exist in the datasets provided by Apache and NASA.

To address the above issues, researchers have proposed many methods that have been proven to be effective in improving class imbalance, class overlap, or noise problems in

datasets. Oversampling methods have been demonstrated to be effective solutions for class imbalance [14,15]. For example, Chawla et al. [16] proposed the Synthetic Minority Over-sampling Technique (SMOTE), He et al. [17] introduced an Adaptive Synthetic Sampling Method (ADASYN), and Benni et al. [14] proposed an oversampling method called MAHAKIL. Gong et al. [18] presented a method based on clustering and oversampling to tackle class imbalance and noise issues. Kim et al. [19] noted that many software defect prediction datasets contain noise and proposed a method based on CLNI to identify and remove noise. Alan et al. [20] introduced a method that identifies outlier noise based on distance thresholds and class labels. Shi et al. [21] proposed a method called US-PONR, which first removes overlapping samples and then performs oversampling and noise handling to address class overlap, class imbalance, and noise problems.

Most of these studies have only considered one or two issues among class imbalance, class overlap, and noise [1,22,23], or have addressed class overlap or noise rather than oversampling [18,24]. Research has shown that class imbalance, class overlap, and noise problems can all have adverse effects on software defect prediction models [2,3,6]. Additionally, addressing class overlap and noise data generated after oversampling can also help further improve model performance [25,26]. Therefore, we believe that all three common issues—class imbalance, class overlap, and noise—should be considered in the process of software defect prediction, and that the overlap and noise issues resulting from data rebalancing through oversampling should also be taken into account.

In summary, we propose a method called AS-KDENN (based on oversampling, kernel density, and nearest neighbor algorithms), which can simultaneously improve the impact of class imbalance, noise, and class overlap issues on classifiers. KDENN is a method we introduced that can handle both noise and class overlap simultaneously. It takes into account both the distance and local density information of samples when addressing noise and class overlap issues.

The main contributions of this paper are as follows:

1. We propose a software defect prediction method, AS-KDENN, which can simultaneously address class imbalance, class overlap, and noise issues. The experimental results demonstrate that our proposed method can effectively improve the performance of software defect prediction (SDP).
2. We introduce a method called KDENN that can simultaneously handle noise and class overlap issues, and the experimental results show that the KDENN method can effectively improve the class overlap and noise problems generated after synthetic sampling.
3. To illustrate the effectiveness of our proposed method, a comprehensive evaluation of the AS-KDENN method was conducted on 19 datasets, and a comparative analysis was performed with commonly used methods that combine class imbalance with handling class overlap or noise.

Through our experimental analysis, we found that the proposed AS-KDENN method can simultaneously improve class imbalance, noise, and class overlap issues, effectively enhancing the performance of software defect prediction models. The software defect prediction models show improvements in the Recall, G-mean, and Balanced accuracy metrics compared to other algorithms.

The structure of the rest of the content in this paper is arranged as follows: Section 2 introduces related research; Section 3 presents the basic framework of our proposed AS-KDENN method and the process for handling class imbalance, class overlap, and noise; Section 4 discusses the research questions and experimental settings, the datasets used, and the model evaluation methods; Section 5 analyzes the experimental results; and Section 6 provides a conclusion for the paper.

2. Related Work

Class imbalance, class overlap, and noise have been proven to be prevalent issues in software defect prediction datasets. Research has shown that these problems can adversely affect classification models [19,27–29]. Therefore, these issues should be considered as much as possible during the software defect prediction process to achieve better classification results.

2.1. Class Imbalance

In software defect datasets, non-defective instances typically far outnumber defective ones, leading to a significant disparity between the two classes and resulting in class imbalance issues [7–9]. Most machine learning algorithms tend to produce results that are biased toward the majority class when handling imbalanced datasets, leading to classification errors [30], which is undesirable. Class imbalance is one of the primary reasons for the poor performance of software defect prediction models [2,31].

Researchers have proposed many methods to address the class imbalance problem. Through our investigation, we found that the majority of researchers adopt data-level rebalancing methods, primarily using undersampling and oversampling techniques to achieve a balance between majority and minority classes. Undersampling methods typically achieve rebalancing by reducing the majority class, but this process may delete instances that are useful for predicting defects. Therefore, oversampling is often more effective and more widely used than undersampling [14,32]. This paper also employs oversampling techniques to address the class imbalance problem.

Common oversampling techniques include SMOTE [16], ADASYN [17], ROS (Random Over Sampling), and Borderline-SMOTE [33]. Oversampling techniques are methods used to achieve rebalancing by increasing the number of minority class samples. However, due to the limitations of oversampling techniques, they may also introduce new problems, such as exacerbating class overlap and noise issues [15,25], which can adversely affect the model. To mitigate the impact of noise and class overlap issues on software defect prediction models after oversampling, Vo et al. [15] proposed a noise-adaptive synthetic oversampling technique (NASOTECH) to address class imbalance and noise in samples. Arafa et al. [34] introduced a method called RN-SMOTE, which uses DBSCAN to remove corresponding noise after applying SMOTE oversampling, thereby improving the noise effects generated post-oversampling. Yang et al. [35] proposed a synthetic sampling method named SD-KMSMOTE, which reduces the overlap of synthetic samples by filtering noise preprocessing and considering the class information for neighboring samples.

2.2. Class Overlap

When multiple class instances exhibit certain feature similarities and share a region in space, class overlap occurs. Class overlap is also an important factor affecting the performance of classifiers [3,24,28]. Research by Gong et al. [3] indicates that many software defect prediction datasets inherently exhibit varying degrees of class overlap issues. Azhar et al. [26] found that the use of oversampling methods can introduce new class overlap problems. To demonstrate this, we randomly generated the imbalanced dataset shown in Figure 1, and applied the ROS, ADASYN, and SMOTE methods of oversampling to balance the positive and negative samples. From Figures 2–4, it can be seen that new class overlap and noise issues were introduced after oversampling. Therefore, while software defect prediction data itself possesses class overlap and noise problems, the application of oversampling methods can also lead to new and varying degrees of class overlap or noise issues.

To address the class overlap problem, Chen et al. [24] employed the Neighborhood Cleaning Rule (NCL) method. NCL manages class overlap by identifying the K-nearest neighbors for each minority class instance and then removing certain negative samples based on the positive-to-negative sample ratio to eliminate overlapping instances. Gong et al. [36] introduced the Improved K-Means Combined with Class Association (IKMCCA) method, which considers the positive and negative sample ratio within each cluster when removing overlapping instances. Goel et al. [37] proposed the CLARANSCA method, designed to address both class overlap and imbalance issues. The core of CLARANSCA in tackling class overlap lies in observing the positive-to-negative sample ratio within each cluster. If the proportion of positive samples exceeds the negative ones, the negative samples are regarded as instances of overlapping and are removed; conversely, if negative samples outnumber positive ones, the positive samples are treated as instances of overlapping and are deleted.

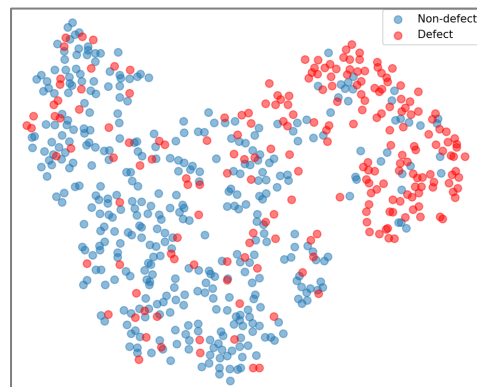


Figure 1. Before oversampling.

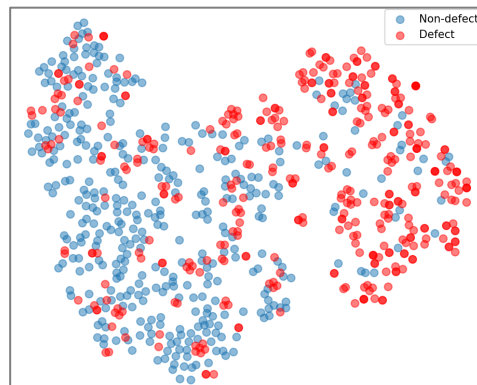


Figure 2. Random oversampling.

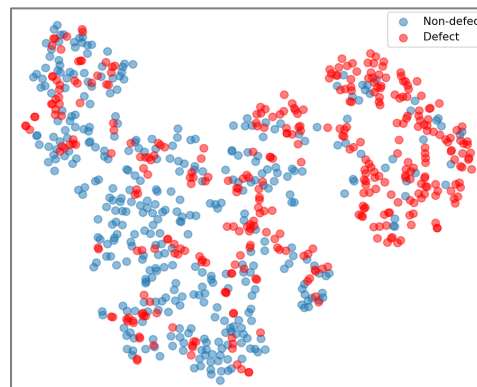


Figure 3. ADASYN oversampling.

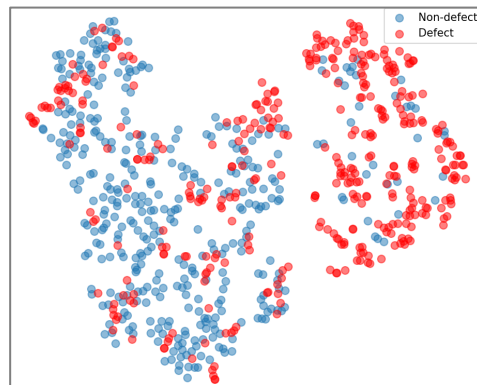


Figure 4. SMOTE oversampling.

2.3. Data Noise

Noise in software defect datasets is often attributed to mislabeled or hidden defects within the modules [5,6]. Ahluwalia et al. [12] examined data from 282 versions of six open-source projects, demonstrating that most datasets suffer from missing defects. Herzig et al. [6] found that among over 7000 issues collected by online defect management systems, more than 40% were incorrectly labeled. Jimenez et al. [11] showed that noise negatively impacts classifiers, leading to erroneous results. Bernhardt et al. [38] demonstrated that cleaning the noise can mitigate the negative effects it may have on model training and evaluation. Oversampling techniques, while generating new samples, may introduce additional noise when the original samples are noisy or due to the method itself [25,34,39]. These noisy data, incorrectly labeled or undiscovered, can be considered outliers in the feature space of non-belonging categories. Stefanowski [40] indicated that these rare or outlier instances can also affect model performance in imbalanced datasets.

To handle the noise in software defect datasets, Tang et al. [41] proposed a K-means-based noise identification method, which was effectively validated using NASA-related datasets. The basic principle involves clustering the dataset and attempting to identify small clusters with fewer data points. Data points in these small clusters that are far from the centroids of the larger clusters are then defined as noise. Kim et al. [19] used the CLNI algorithm to deal with noise. The CLNI algorithm generates a nearest neighbor list for each instance by calculating the Euclidean distances of neighboring instances and then recognizing instances that have labels which are significantly different from those of their neighbors; these instances are considered as noise. In the study by Batista et al. [42], after applying SMOTE oversampling, the ENN algorithm was used to remove noise in order to reduce the impact of noise on software defect prediction models.

3. Materials and Methods

3.1. Method Overview

The AS-KDENN method we propose combines oversampling, class overlap, and noise handling. It not only uses oversampling techniques to address class imbalance but also employs the KDENN method based on kernel density (KDE) and distances to manage noise and class overlap data. This approach can mitigate the impact of noise and class overlap induced by oversampling on classification models, thereby enhancing the overall performance of software defect prediction models.

Our method consists of two main stages: (1) the first stage improves class imbalance through sampling methods; (2) the second stage addresses noise and class overlap in the oversampled data. In the oversampling stage, we use the ADASYN to synthesize more minority class instances, balancing positive and negative samples to resolve the class imbalance issue. In the noise and class overlap processing stage, we apply our proposed KDENN algorithm to handle noise and class overlap in the oversampled data. The KDENN algorithm is a newly proposed method that combines local density and nearest neighbors.

The overall framework of the method is illustrated in Figure 5.

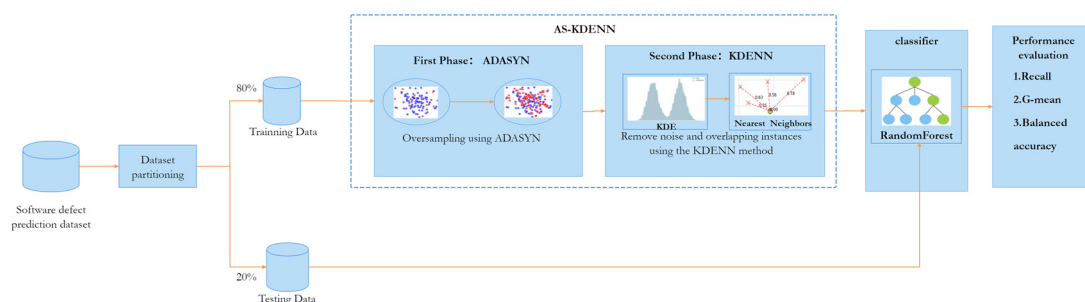


Figure 5. AS-KDENN method framework.

3.2. Oversampling

Oversampling is a technique used to rebalance positive and negative samples by increasing the number of minority class instances. In our method framework, we selected ADASYN as our oversampling algorithm. Due to their simplicity, efficiency, and classifier-independence, oversampling methods like SMOTE, ADASYN, and ROS have been widely used to address data imbalance issues [25,43]. Many researchers have utilized these methods to improve classifier performance [44,45].

The core principle of the SMOTE method [16] is to use the KNN algorithm to find the nearest neighbors based on Euclidean distance for a specified K value, which is usually set to 5. Then, a nearest neighbor is randomly selected, and a sample is randomly inserted between the selected point and this nearest neighbor. The specific method is as follows:

$$x_{\text{new}} = x_i + (\hat{x}_i - x_i) \times \delta \quad (1)$$

$x_i \in s_{\text{min}}$ is a minority class sample, and $\hat{x}_i$ is one of the neighbors of x_i . $\delta \in [0, 1]$ is a random number. Figure 6 illustrates a randomly generated synthetic sample:

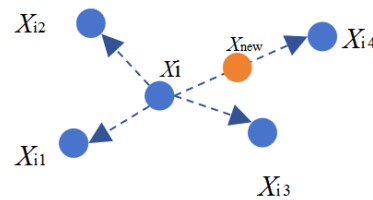


Figure 6. SMOTE synthetic sample generation.

The ADASYN oversampling method [17] is an adaptive synthetic oversampling technique. ADASYN assigns a weight to each minority class sample, which also indicates the number of synthetic samples to be generated. Compared to SMOTE, ADASYN considers the proportion of majority class samples among the nearest neighbors, allowing for more synthetic samples to be generated in sparser regions. This helps to better capture the characteristics of the minority class and improve the recognition of minority class samples. The main process of generating synthetic samples using ADASYN is as follows:

Step1: Calculate the number of synthetic samples needed:

$$G = (m_l - m_s) \times \beta \quad (2)$$

where m_l is the number of majority class samples, m_s is the number of minority class samples, and $\beta \in [0, 1]$ is a random number. If β equals 1, the ratio of positive to negative samples after sampling will be 1:1.

Step2: Calculate the proportion of majority class samples among the K-nearest neighbors. The formula for this is as follows:

$$r_i = \Delta i / K \quad (3)$$

where Δi is the number of majority class samples among the K-nearest neighbors for each minority class sample i , with $i = 1, 2, 3, \dots, m_s$.

Step3: Normalize

$$\hat{r}_i = r_i / \sum_{i=1}^{m_s} r_i \quad (4)$$

Step4: Based on the sample weights, calculate the number of new samples that need to be generated for each minority class sample. The formula for this is as follows:

$$g = \hat{r}_i \times G \quad (5)$$

Step5: Calculate the number of samples to be generated for each minority sample based on g , and synthesize new samples using the following formula:

$$s_i = x_i + (x_{zi} - x_i) \times \lambda \quad (6)$$

where s_i is the synthetic sample, x_i is the i -th sample from the minority class, x_{zi} is a randomly selected minority class sample from the K -nearest neighbors of x_i , and $\lambda \in [0, 1]$ is a random number. This approach generates synthetic samples by interpolating between a minority sample and one of its nearest neighbors, helping to balance the class distribution.

Random Oversampling (ROS) primarily involves selecting a certain number of samples from the minority class and either duplicating them directly or adding slight transformations during duplication to enhance the data. This approach aims to increase the number of minority class samples, thereby addressing the class imbalance problem. In the context of random oversampling, let N_1 represent the number of minority class samples, N_0 represent the number of majority class samples, and k be the oversampling factor. The total number of samples after oversampling will be $N_0 + k \times N_1$. Typically, k is set to 1 or 2, meaning the minority class samples are either duplicated once or twice to achieve the desired balance.

Through the above research, we can also observe that ADASYN, compared to SMOTE and ROS, takes into account the proportion of the majority class among the nearest neighbors when synthesizing samples. This allows ADASYN to generate more diverse and reasonable minority class samples. This consideration is the reason why we chose ADASYN as the oversampling method for this study.

3.3. Noise and Class Overlap Handling

Through the studies in Section 2.2, we found that many class overlap and noise detection methods are based on distance, without considering the local density of instances. Relying solely on distance might lead to certain misjudgments. For example, when using Edited Nearest Neighbors (ENN) to remove overlapping instances, consider a negative sample T as shown in Figure 7. If we are following the nearest neighbor method (assuming $K = 5$), and the five nearest neighbors of point T mostly contain negative samples, we might deem points A and B as overlapping instances for T and consequently remove these two positive samples as noise. This is clearly unreasonable since A and B , from a spatial perspective, belong to the normal minority class of samples. The same issue can occur when removing negative samples under similar circumstances. This highlights the importance of considering the local density and distribution of samples rather than just their proximity, which can prevent the erroneous removal of valid minority class samples that might appear to be noise or overlapping purely based on distance.

To address this, we propose a method that combines Kernel Density Estimation (KDE) and K -nearest neighbors. This method not only uses nearest neighbors to identify potentially overlapping instances but also considers the ratio of positive to negative samples while taking into account their density distribution when assessing overlapping instances. The KDENN method consists of two steps: the first step involves using the KDE algorithm to remove noisy data, and the second step combines KDE with K -nearest neighbors to eliminate class overlapping data based on the results of the first step.

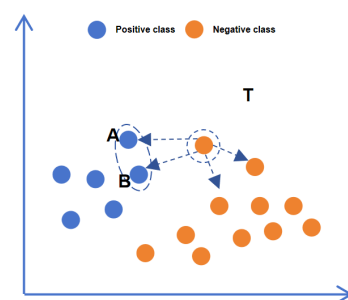


Figure 7. Example of nearest neighbor overlap removal.

Considering that the distribution of normal instances within the same class should exhibit continuity or clustering characteristics, we treat the sparse classes estimated by

KDE as noise to be removed [46]. In addressing the overlap issue, we not only consider the distance between classes but also pay attention to the local density of the classes.

For each given sample, there exists a density function in both the minority and majority classes, denoted as $PKDE_{min}$ and KDE_{max} , respectively. We denote their ratio as Δk_{ratio} . When Δk_{ratio} is greater than 1, it indicates that this instance is closer to the minority class region; when it is less than 1, it indicates that the instance is closer to the majority class region. In using the nearest neighbor algorithm to remove overlapping instances, we typically determine which instances are overlapping based solely on the ratio of positive to negative samples and remove either positive or negative samples. In our method, we first find the K-nearest neighbors using the nearest neighbor algorithm. If the ratio of negative samples is greater than that of positive samples, we do not directly remove all positive samples; instead, we consider the density ratio of each positive sample in the minority and majority classes to decide whether to classify it as an overlapping class for deletion. Since we are more concerned with positive samples, which represent the minority class, if the proportion of positive samples in a given nearest neighbor list is greater than that of negative samples, we will remove all negative samples as overlapping classes directly. The algorithm process is illustrated in Algorithm 1.

Algorithm 1. KDENN handles noise and class overlapping data

Input: Training data T

threshold for the positive-to-negative density ratio:th

Begin

Step1: Using KDE to remove noisy data

1: Divide the training data T into non-defective data T^- and defective data T^+

2: Calculate the kernel density for each sample in dataset T^- , denoted as KDE_{max}

3: Calculate the kernel density for each sample in dataset T^+ , denoted as KDE_{min}

4: In the T^- dataset, remove the data points with a probability density below 2% as noise to obtain a new dataset P^-

5: In the T^+ dataset, remove the data points with a probability density below 2% as noise to obtain a new dataset P^+

6: Merge datasets P^- and P^+ to obtain dataset P after removing noise

Step2: Removing class overlapping data based on KDE and K-nearest neighbors

1: Calculate the kernel density for each sample in dataset P^- , denoted as $PKDE_{max}$

2: Calculate the kernel density for each sample in dataset P^+ , denoted as $PKDE_{min}$

3: Find the ten nearest neighbors for each sample, denoted as N_x

4: Calculate the ratio of positive to negative samples in N_x , denoted as Δr_{pn}

5: For each sample $X \in N_x$

6: IF $\Delta r_{pn} > 1$:

delete all negative sample data from this cluster

7: Calculate Δk_{ratio} for X, $\Delta k_{ratio} = PKDE_{max} / PKDE_{min}$

8: IF $\Delta r_{pn} < 1$ and $\Delta k_{ratio} > th$

delete this positive sample

9: End for

10: Output: Obtain the final dataset T' after removing noise and overlapping instances

4. Experimental Design

This section provides a detailed description of the research problem, the software defect prediction datasets used, and the model evaluation metrics.

4.1. Research Problem and Experimental Process

As mentioned above, our goal is to simultaneously address class imbalance, noise, and class overlap issues through our proposed AS-KDENN method in order to enhance the effectiveness of software defect prediction. In this process, we utilize the ADASYN for oversampling the data to resolve the data imbalance issue, and we employ our proposed KDENN method to comprehensively consider the noise and class overlap problems

introduced by the original dataset and the oversampling method. To demonstrate the effectiveness of our proposed method, we have formulated the following research questions:

RQ1: Is our proposed AS-KDENN more effective than other common oversampling methods for handling noise or class overlap?

RQ2: Is our proposed data noise and overlap removal method (KDENN) equally as effective as other oversampling methods?

RQ3: How efficient is our proposed method in the software defect prediction process?

RQ1 is the most critical question in our research. Through RQ1, we can determine whether our proposed AS-KDENN method is more effective than the selected baseline methods. To achieve this, we chose four common methods for oversampling that consider noise or class overlap: SMOTE-ENN, SMOTE-TOMEK, Borderline-SMOTE, and ADASYN-TOMEK. We will compare the performances of these four methods with the AS-KDENN method across 18 datasets to obtain our research results.

The main purpose of RQ2 is to observe whether KDENN, as a method for handling class overlap and noise, is also effective for other oversampling methods. To this end, we evaluated the effect of using the KDENN method before and after on commonly used oversampling algorithms: ROS, ADASYN, and SMOTE across 18 datasets.

The primary goal of RQ3 is to assess the efficiency of our designed AS-KDENN method across all datasets. Since our method utilizes KDE for kernel density estimation and the K-nearest neighbors algorithm, which requires traversing each instance to calculate its related density and search for nearest neighbors, it may lead to decreased performance efficiency on large datasets. Therefore, we will statistically analyze the execution time of each method across each dataset to identify any significant differences in performance.

4.2. Experimental Setup

In this study, we divided the dataset in an 80:20 ratio, with 80% allocated for training data and 20% for testing data. Since the dataset division is random, to ensure the reliability of the experimental results, we conducted 20 repeated training and testing sessions for the AS-KDENN method and recorded its average performance for each.

During the experimental comparison, to minimize the bias introduced by the methods, we selected comparison methods that are commonly used and implemented using the sklearn and imblearn libraries [18,21].

4.3. Datasets

In this paper, to demonstrate that the model has better generalization capabilities on the dataset, we used a total of 19 projects from four datasets: PROMISE [47], AEEEM [48], NASA [49], and Relink [50]. These datasets are publicly available and are written in different programming languages. They are commonly used benchmark datasets in the field of software defect prediction research [51], and they are also frequently utilized by other researchers to study issues such as class imbalance, class overlap, and noise [18,23,36]. These datasets can be used for publicly evaluating the performance of software defect prediction classifiers.

We selected our experimental projects from these four datasets primarily based on the following two criteria: (1) Based on our experience, the defect rate generally does not exceed 40%, so datasets with a defect rate greater than 40% are not considered, as such the data are more meaningful in practice. (2) The size of and degree of imbalance in the datasets need to have a certain diversity to facilitate our observation of the effectiveness of the methods across different datasets. Following these two principles, we selected a total of 19 projects and conducted extensive experiments on them.

The statistical information about the datasets we selected, including the number of instances (#Instances), the number of features (#Metrics), the number of faulty instances (#Faulty Instances), the Defective Ratio, and the Imbalance Ratio, is shown in Table 1.

Table 1. Statistics of software defect prediction datasets.

Dataset	Project	#Instances	#Metrics	#Faulty Instances	Defective Ratio	Imbalance Ratio
Promise	prop-1.csv	18,471	20	2738	14.80%	5.75
	prop-2.csv	23,014	20	2431	10.60%	8.47
	prop-3.csv	10,274	20	1180	11.50%	7.71
	prop-4.csv	8718	20	840	9.60%	9.38
	prop-5.csv	8516	20	1299	15.30%	5.56
	ivy-2.0.csv	352	20	40	11.36%	7.8
	jedit-4.2.csv	367	20	48	13.08%	6.65
Relink	safe.csv	56	26	22	39.29%	1.55
	zxing.csv	399	26	118	29.57%	2.38
NASA	MW1.csv	253	37	27	10.67%	8.37
	PC1.csv	705	37	61	8.70%	10.56
	PC3.csv	1077	37	134	12.40%	7.04
	PC4.csv	1458	37	178	12.20%	7.19
	PC5.csv	1711	38	471	27.50%	2.63
AEEEM	xerces-1.2.csv	440	20	71	16.14%	5.2
	PDE.csv	1497	15	209	14.00%	6.16
	xalan-2.4.csv	723	20	110	15.20%	5.57
	jdt.csv	997	15	206	20.70%	3.84
	mylyn.csv	1862	15	245	13.20%	6.6

4.4. Model Evaluation

When addressing imbalance issues, the accurate detection of minority class instances is crucial. This is often evaluated based on Sensitivity, also known as the True Positive Rate (TPR) or Recall, which is measured as shown in Equation (7), where TP and FN represent True Positives and False Negatives, respectively. Recall is one of the commonly used metrics in the software defect prediction process [14,17,18].

Balanced accuracy is the arithmetic mean of the accuracy of each class, as shown in Equation (8). It is also referred to as balanced average accuracy or average accuracy [28]. When class imbalance is high, the negative class accuracy can be significantly misleading. For example, in an imbalanced dataset with 100 instances, if 80 negative instances are perfectly classified but all 20 positive instances are misclassified, the calculated accuracy would still be as high as 80%, which could mislead the results. Therefore, we typically use Balanced accuracy to replace traditional accuracy, and it is one of the most commonly used metrics for addressing imbalance issues [24].

Another commonly used metric for evaluating overall model performance is G-mean [52]. It is the geometric mean of sensitivity and specificity, as shown in Equation (9). From the literature review, it is clear that G-mean is a widely used metric for assessing the comprehensive performance of models. It takes into account the model's performance on both the positive and negative classes, aiming to balance the True Positive Rate (Recall) and True Negative Rate (Specificity). It is widely used in various software defect prediction studies [29,34].

Our research aims to propose a software defect prediction (SDP) method that simultaneously considers class imbalance, class overlap, and noise in order to discover as many bugs as possible. In practical applications, an effective software defect prediction model should be able to identify as many potential bugs in the system as possible. This is crucial to avoid incurring additional costs later on for bug fixes. Therefore, we chose Recall as our

primary performance metric. To also reflect the recall situation of positive and negative samples on imbalanced datasets, we selected Balanced accuracy as our second performance metric. Given that the software defect prediction dataset is imbalanced, we chose G-mean as our third performance evaluation metric to better assess our model's overall performance on imbalanced datasets.

1. Recall: Also known as sensitivity, Recall reflects the ability to correctly predict defects. It is defined as

$$\text{Recall} = \frac{TP}{TP + FN} \quad (7)$$

where TP represents the number of defective instances correctly identified, and FN represents the number of non-defective instances incorrectly classified as defective

2. Geometric Mean (G-mean): G-mean is a comprehensive evaluation metric often used to assess the performance of classifiers on imbalanced datasets. It reflects the classifier's ability to correctly identify both positive and negative samples, and provides a better indication of model performance compared to simple classification accuracy. The calculation formula is as follows:

$$\text{G-mean} = \sqrt{\frac{TP}{TP + FN} \times \frac{TN}{TN + FP}} \quad (8)$$

3. Balanced Accuracy (Ba): In binary classification, Balanced accuracy is the arithmetic mean of the accuracy of positive and negative samples. When there is class imbalance, the accuracy of the majority class can be misleading. Using Balanced accuracy helps avoid exaggerated evaluations based on a single metric for imbalanced datasets. It is defined as

$$\text{Balanced accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right) \quad (9)$$

To better observe the differences between our proposed method and other methods, we chose the Wilcoxon signed-rank test to calculate the p -value [53]. The Wilcoxon signed-rank test is a non-parametric test commonly used to determine whether there is a statistically significant difference between two groups of data. At a 95% confidence level, if the p -value is less than 0.05, it can be considered that there is a statistically significant difference between the two methods. Additionally, we also used Cliff's Delta [54] to quantify the magnitude of the difference. Cliff's Delta is a non-parametric effect size measure used to assess the significance of the difference between two groups of data, with a range of values from $[-1, 1]$. A positive value indicates that the proposed method outperforms the baseline, while a negative value indicates that the baseline method outperforms the proposed method. By using the Wilcoxon test and Cliff's Delta, we can better determine whether our proposed method has a substantial improvement over the baseline method.

5. Results and Discussion

This section will address the research questions posed in Section 4.1 by analyzing the experimental results.

5.1. Q1: Is Our Proposed AS-KDENN More Effective than Other Common Oversampling Methods That Handle Noise or Class Overlap?

(A) Comparison Methods

We compared the AS-KDENN method with common methods that address noise, class overlap, or class imbalance. We selected four commonly used baseline comparison methods: SMOTE-ENN, SMOTE-TOMEK, Borderline-SMOTE, and ADASYN-TOMEK. These methods have also been shown by researchers to effectively improve issues related

to class imbalance, class overlap, or noise, and have been used as baseline comparison methods in many studies [18,21,22].

(B) Statistical Analysis

To visually compare the performance of these methods across 19 datasets, we first plotted the Recall and G-mean performances as box plots, as shown in Figure 8. From Figure 8, we can intuitively see that the median of the AS-KDENN method is higher than the other four methods in all three metrics, which indicates its performance is superior to the other four methods to some extent. To further demonstrate the superiority of the AS-KDENN method, we employed the non-parametric Wilcoxon test to examine the significance of performance differences among the five methods, as shown in Table 2. At a 95% confidence level, the p -values were all less than 0.05. Tables 3–6 present the Cliff’s Delta effect size calculations. Evaluating these results collectively, we can see that the performance of the AS-KDENN method surpasses that of the other four methods.

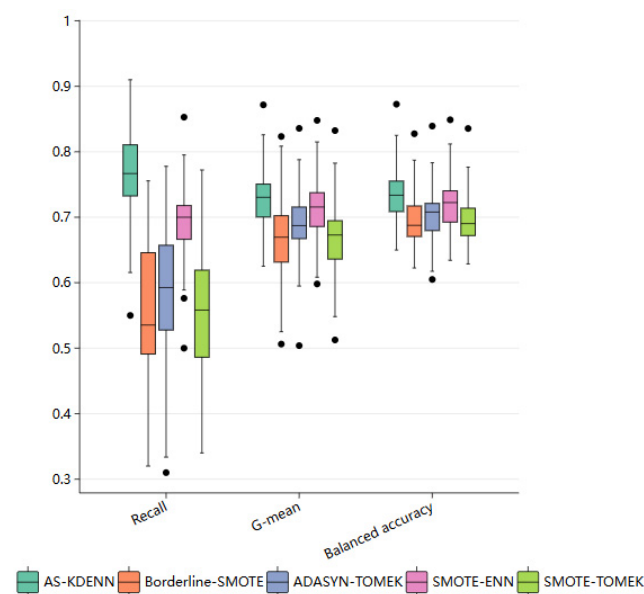


Figure 8. Comparison of AS-KDENN with other oversampling methods combined with class overlap or noise.

Table 2. Wilcoxon signed-rank test results p value.

Method	Recall	G-Mean	Ba
AS-KDENN vs. Borderline-SMOTE	3.81×10^{-6}	7.25×10^{-5}	2.67×10^{-5}
AS-KDENN vs. ADASYN-TOMEK	3.81×10^{-6}	1.64×10^{-4}	1.64×10^{-3}
AS-KDENN vs. SMOTE-ENN	1.91×10^{-5}	1.4×10^{-2}	1.69×10^{-3}
AS-KDENN vs. SMOTE-TOMEK	3.81×10^{-6}	5.34×10^{-5}	1.69×10^{-3}

Table 3. Cliff’s Delta effect of AS-KDENN compared to ADASYN-TOMEK method.

Project	Recall	G-Mean	Ba
jdt.csv	0.77(large)	0.0925(negligible)	0.005(negligible)
mw1.csv	0.6775(large)	0.485(large)	0.3025(small)
mylyn.csv	0.885(large)	0.635(large)	0.565(large)
ivy-2.0.csv	0.76(large)	0.5875(large)	0.4825(large)
jedit-4.2.csv	0.68(large)	0.505(large)	0.41(medium)

Table 3. *Cont.*

Project	Recall	G-Mean	Ba
safe.csv	0.5075(large)	0.1(negligible)	0.14(negligible)
xerces-1.2.csv	0.725(large)	0.37(medium)	0.3325(medium)
new_zxing.csv	0.9825(large)	−0.2125(small)	−0.05(negligible)
PC1.csv	0.75(large)	0.575(large)	0.47(medium)
PC3.csv	0.88(large)	0.5825(large)	0.53(large)
PC4.csv	0.87(large)	0.55(large)	0.565(large)
PC5.csv	1.0(large)	−0.3(small)	−0.005(negligible)
PDE.csv	1.0(large)	0.91(large)	0.82(large)
xalan-2.4.csv	0.87(large)	0.6925(large)	0.6125(large)
prop-1.csv	1.0(large)	0.24(small)	0.34(medium)
prop-2.csv	1.0(large)	−0.175(small)	−0.205(small)
prop-3.csv	1.0(large)	0.605(large)	0.61(large)
prop-4.csv	0.99(large)	0.785(large)	0.63(large)
prop-5.csv	1.0(large)	0.76(large)	0.885(large)

Table 4. Cliff's Delta effect of AS-KDENN compared to SMOTE-ENN method.

Project	Recall	G-Mean	Ba
jdt.csv	0.575(large)	−0.08(negligible)	−0.11(negligible)
mw1.csv	0.12(negligible)	0.0925(negligible)	0.0975(negligible)
mylyn.csv	0.8475(large)	0.6625(large)	0.64(large)
ivy-2.0.csv	−0.0225(negligible)	0.01(negligible)	0.01(negligible)
jedit-4.2.csv	0.1025(negligible)	−0.01(negligible)	−0.0175(negligible)
safe.csv	0.4475(medium)	0.22(small)	0.245(small)
xerces-1.2.csv	0.23(small)	0.2275(small)	0.2125(small)
zxing.csv	0.6575(large)	−0.1525(small)	0.065(negligible)
PC1.csv	−0.1675(small)	−0.1225(negligible)	−0.115(negligible)
PC3.csv	0.29(small)	0.125(negligible)	0.1375(negligible)
PC4.csv	0.5375(large)	0.37(medium)	0.4(medium)
PC5.csv	0.735(large)	−0.2275(small)	0.13(negligible)
PDE.csv	0.4825(large)	0.15(small)	0.14(negligible)
xalan-2.4.csv	0.165(small)	−0.005(negligible)	−0.005(negligible)
prop-1.csv	1.0(large)	0.44(medium)	0.45(medium)
prop-2.csv	1.0(large)	0.485(large)	0.505(large)
prop-3.csv	1.0(large)	0.78(large)	0.78(large)
prop-4.csv	0.975(large)	0.8(large)	0.7(large)
prop-5.csv	1.0(large)	0.3975(medium)	0.635(large)

Table 5. Cliff’s Delta effect of AS-KDENN compared to SMOTE-TOMEK method.

Project	Recall	G-Mean	Ba
jdt.csv	0.89(large)	0.1(negligible)	−0.085(negligible)
mw1.csv	0.5375(large)	0.3275(small)	0.1875(small)
mylyn.csv	0.9425(large)	0.68(large)	0.635(large)
ivy-2.0.csv	0.7825(large)	0.63(large)	0.455(medium)
jedit-4.2.csv	0.8075(large)	0.6825(large)	0.605(large)
safe.csv	0.5825(large)	0.385(medium)	0.4925(large)
xerces-1.2.csv	0.705(large)	0.29(small)	0.255(small)
new_zxing.csv	0.985(large)	−0.0975(negligible)	0.035(negligible)
PC1.csv	0.7175(large)	0.5675(large)	0.47(medium)
PC3.csv	0.9325(large)	0.7(large)	0.6525(large)
PC4.csv	0.93(large)	0.675(large)	0.68(large)
PC5.csv	1.0(large)	−0.04(negligible)	0.14(negligible)
PDE.csv	1.0(large)	0.85(large)	0.715(large)
xalan-2.4.csv	0.94(large)	0.8475(large)	0.775(large)
prop-1.csv	1.0(large)	0.315(small)	0.205(small)
prop-2.csv	1.0(large)	0.185(small)	0.105(negligible)
prop-3.csv	1.0(large)	0.76(large)	0.755(large)
prop-4.csv	1.0(large)	0.915(large)	0.83(large)
prop-5.csv	1.0(large)	0.77(large)	0.875(large)

Table 6. Cliff’s Delta effect of AS-KDENN compared to Borderline-SMOTE method.

Project	Recall	G-Mean	Ba
jdt.csv	0.8425(large)	0.105(negligible)	0.005(negligible)
mw1.csv	0.6225(large)	0.3825(medium)	0.3025(small)
mylyn.csv	0.94(large)	0.475(large)	0.565(large)
ivy-2.0.csv	0.7(large)	0.495(large)	0.4825(large)
jedit-4.2.csv	0.705(large)	0.495(large)	0.41(medium)
safe.csv	0.5925(large)	0.28(small)	0.14(negligible)
xerces-1.2.csv	0.805(large)	0.6025(large)	0.3325(medium)
new_zxing.csv	0.965(large)	−0.0825(negligible)	−0.05(negligible)
PC1.csv	0.9075(large)	0.82(large)	0.47(medium)
PC3.csv	0.9575(large)	0.75(large)	0.53(large)
PC4.csv	0.9375(large)	0.66(large)	0.565(large)
PC5.csv	0.9975(large)	−0.25(small)	−0.005(negligible)
PDE.csv	1.0(large)	0.785(large)	0.82(large)
xalan-2.4.csv	0.84(large)	0.605(large)	0.6125(large)
prop-1.csv	1.0(large)	0.485(large)	0.34(medium)
prop-2.csv	1.0(large)	0.57(large)	−0.205(small)
prop-3.csv	1.0(large)	0.76(large)	0.61(large)
prop-4.csv	0.99(large)	0.855(large)	0.63(large)
prop-5.csv	1.0(large)	0.91(large)	0.885(large)

(C) Result Analysis

We analyzed the experimental results as follows: First, the issues of class imbalance, noise, and class overlap in software defect prediction datasets have been shown by researchers to have a significant impact on software defect prediction classifiers [15,26,55]. Our method addresses these three issues more comprehensively than others. Second, we created clear definitions of noise and class overlap during our research process and proposed more reasonable distance- and density-based approaches for handling noise and class overlap. Therefore, our AS-KDENN method achieved better prediction results compared to the other four methods, particularly showing a significant advantage in recall rate. This is also what we expect: to discover as much defect data as possible.

5.2. Q2: Does Using the KDENN Method for Handling Class Overlap and Noise on Oversampled Data Help Improve the Performance of Software Defect Prediction Models?

(A) Comparison Method

Through our research in the previous sections, we found that both the oversampled and original software defect prediction datasets exhibit certain levels of noise and overlap. To mitigate the effects of overlap and noise on the classifier, we propose the KDENN method to handle the noise and class overlap in the oversampled data. To demonstrate the effectiveness of the KDENN method, we use three common oversampling methods—ROS, SMOTE, and ADASYN—as baseline methods. The results of their oversampled data were then processed using KDENN to treat the noise and class overlap. Finally, we observe the changes in classifier performance before and after KDENN processing to prove its effectiveness.

(B) Statistical Analysis

Figures 9–11 report the performance results of the three oversampling methods without noise and class overlap treatment, as well as the results after applying the KDENN method. Through the performance of the software defect prediction model on the Recall, G-mean, and Balanced accuracy metrics, it can be seen that the KDENN method suppresses the noise and class overlap issues in the oversampled data, enhancing classification performance. To further analyze the advantages of using KDENN, we applied the Wilcoxon signed-rank test to analyze the significance of the performance of the three methods. As shown in Table 7, at a 95% confidence level, the p -value is significantly less than 0.05, indicating that the performance improvement after using KDENN is evident.

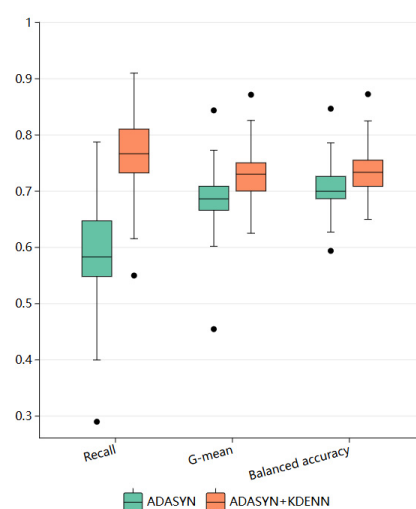


Figure 9. Comparison of ADASYN vs. ADASYN+KDENN.

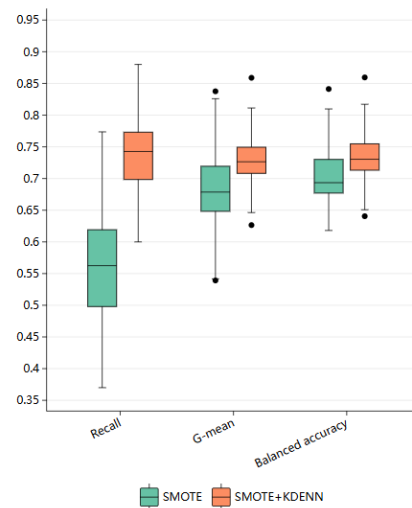


Figure 10. Comparison of SMOTE vs. SMOTE+KDENN.

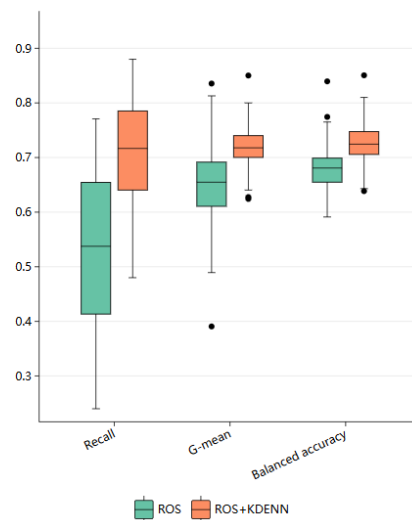


Figure 11. Comparison of ROS vs. ROS+KDENN.

Table 7. Wilcoxon test results for oversampling methods with and without KDENN.

Project	Recall	G-Mean	Ba
Random+KDENN vs. Random	3.81×10^{-6}	2.67×10^{-5}	2.67×10^{-5}
SMOTE+KDENN vs. SMOTE	3.81×10^{-6}	3.81×10^{-6}	3.81×10^{-6}
ADASYN+KDENN vs. ADASYN	3.81×10^{-6}	1.64×10^{-4}	3.81×10^{-5}

(C) Result Analysis

Through the statistical analysis of the experimental results, we can see that, in the 19 datasets we selected, the Recall metric improved across all datasets after using KDENN. The G-mean metric improved in 14 datasets, and the Balanced accuracy metric improved in 12 datasets. Overall, in the imbalanced software defect prediction datasets, using our proposed KDENN method after oversampling effectively helps enhance the performance of the SDP model. We believe that this is primarily because the KDENN method effectively and reasonably eliminates noise and class overlap in the oversampled data, thereby improving the performance of the software defect prediction classifier.

5.3. Q3: How Efficient Is Our Proposed Method in the Software Defect Prediction Process?

(A) Comparison Method

Across the 19 datasets we selected, we conducted a statistical comparison of the execution times of the AS-KDENN method against four other methods: Borderline-SMOTE, SMOTE-ENN, SMOTE-TOMEK, and ADASYN-TOMEK.

(B) Statistical Analysis

Figure 12 reports the execution times of the five methods across the 19 datasets. From this Figure, we can see that on the mylyn, prop-1, prop-2, prop-3, prop-4, and prop-5 projects, the execution time of the AS-KDENN method is significantly higher than that of the other methods. Additionally, as the volume of data increases, the execution time exhibits a certain super-linear growth trend. On other datasets, the execution time of AS-KDENN is roughly twice that of the other methods.

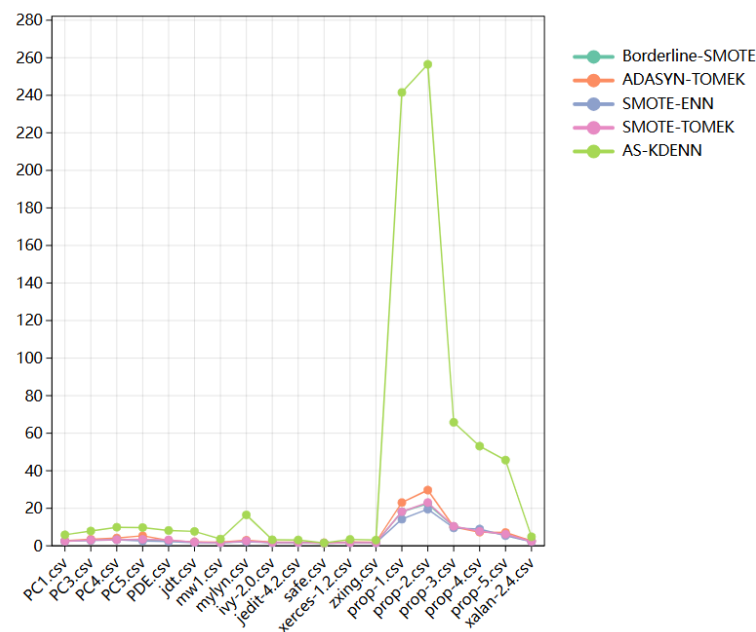


Figure 12. Comparison of execution times.

(C) Result Analysis

We analyze that the main reason for the higher execution time of the AS-KDENN method compared to other methods is the need to perform a significant amount of kernel density calculations and K-nearest neighbor search tasks, which are time-consuming processes. Nonetheless, we believe that our method still holds application value compared to automated testing and traditional manual testing, as the majority of projects we encountered are small-to-medium-sized. Large projects like prop-1 and prop-2 are in the minority, and even the longest execution time reached 256 s. This is still competitive when compared to the time required for automated and manual testing.

6. Conclusions and Future Work

In the field of software defect prediction, issues such as class imbalance, noise, and class overlap can significantly impact the performance of classifiers. This paper proposes the AS-KDENN method, which simultaneously addresses class imbalance, noise, and class overlap issues. Improvements were made to existing distance-based methods in dealing with noise and class overlap by incorporating the local density of overlapping samples. This approach can reduce some misclassifications that are solely based on distance.

We conducted extensive evaluations of the AS-KDENN method on 18 publicly available software defect prediction datasets. The results indicate that using the AS-KDENN

method can enhance the performance of the SDP model in terms of Recall, G-Mean, and Balanced accuracy, considering class imbalance, noise, and class overlap. The KDENN method effectively alleviates the issues of class overlap and imbalance brought about by oversampling techniques. Therefore, we recommend that future software defect prediction efforts comprehensively consider class imbalance, noise, and class overlap issues, particularly when using oversampling methods to address class imbalance.

In our next steps, we plan to first apply AS-KDENN to some commercial projects based on our laboratory's CNAS platform to further evaluate its performance and practicality. Secondly, we aim to combine our method with deep learning techniques for feature selection and replace the existing random forests with ensemble learning classifiers to further enhance model performance. Additionally, we hope to explore the quantitative relationships between the AS-KDENN method and class overlap, noise, and class imbalance.

Author Contributions: Writing—original draft, R.W., F.L. and Y.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original data presented in the study are openly available in Version v1.0 at [10.5281/zenodo.6342327].

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Pelayo, L.; Dick, S. Applying Novel Resampling Strategies to Software Defect Prediction. In Proceedings of the NAFIPS 2007—2007 Annual Meeting of the North American Fuzzy Information Processing Society, San Diego, CA, USA, 24–27 June 2007; pp. 69–72. [\[CrossRef\]](#)
2. Hall, T.; Beecham, S.; Bowes, D.; Gray, D.; Counsell, S. A Systematic Literature Review on Fault Prediction Performance in Software Engineering. *IEEE Trans. Softw. Eng.* **2012**, *38*, 1276–1304. [\[CrossRef\]](#)
3. Gong, L.; Zhang, H.; Zhang, J.; Wei, M.; Huang, Z. A Comprehensive Investigation of the Impact of Class Overlap on Software Defect Prediction. *IEEE Trans. Softw. Eng.* **2023**, *49*, 2440–2458. [\[CrossRef\]](#)
4. Bhandari, K.; Kumar, K.; Sangal, A.L. Data Quality Issues in Software Fault Prediction: A Systematic Literature Review. *Artif. Intell. Rev.* **2023**, *56*, 7839–7908. [\[CrossRef\]](#)
5. Croft, R.; Babar, M.A.; Chen, H. Noisy Label Learning for Security Defects. In *2022 Mining Software Repositories Conference (MSR 2022), Proceedings of the IEEE International Working Conference on Mining Software Repositories*, Pittsburgh, PA, USA, 23–24 May 2022; IEEE Computer Society: Los Alamitos, CA, USA, 2022; pp. 435–447. [\[CrossRef\]](#)
6. Herzig, K.; Just, S.; Zeller, A. It's Not a Bug, It's a Feature: How Misclassification Impacts Bug Prediction. In Proceedings of the 2013 35th International Conference on Software Engineering (ICSE), San Francisco, CA, USA, 18–26 May 2013; pp. 392–401. [\[CrossRef\]](#)
7. Bowes, D.; Hall, T.; Gray, D. DConfusion: A Technique to Allow Cross Study Performance Evaluation of Fault Prediction Studies. *Autom. Softw. Eng.* **2014**, *21*, 287–313. [\[CrossRef\]](#)
8. Menzies, T.; Dekhtyar, A.; Distefano, J.; Greenwald, J. Problems with Precision: A Response to “Comments on ‘Data Mining Static Code Attributes to Learn Defect Predictors’”. *IEEE Trans. Softw. Eng.* **2007**, *33*, 637–640. [\[CrossRef\]](#)
9. Wang, S.; Yao, X. Using Class Imbalance Learning for Software Defect Prediction. *IEEE Trans. Reliab.* **2013**, *62*, 434–443. [\[CrossRef\]](#)
10. Riquelme Santos, J.C.; Ruiz Sánchez, R.; Rodríguez García, D.; Moreno, J. Finding Defective Modules from Highly Unbalanced Datasets. *Actas Talleres Jorn. Ing. Softw. Y Bases Datos* **2008**, *2*, 67–74.
11. Jimenez, M.; Rwemalika, R.; Papadakis, M.; Sarro, F.; Le Traon, Y.; Harman, M. The Importance of Accounting for Real-World Labelling When Predicting Software Vulnerabilities. In Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Tallinn, Estonia, 26–30 August 2019; ESEC/FSE 2019. Association for Computing Machinery: New York, NY, USA, 2019; pp. 695–705. [\[CrossRef\]](#)
12. Ahluwalia, A.; Falessi, D.; Di Penta, M. Snoring: A Noise in Defect Prediction Datasets. In Proceedings of the 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), Montreal, QC, Canada, 25–31 May 2019; pp. 63–67. [\[CrossRef\]](#)
13. Pandey, S.K.; Tripathi, A.K. An Empirical Study toward Dealing with Noise and Class Imbalance Issues in Software Defect Prediction. *Soft Comput.* **2021**, *25*, 13465–13492. [\[CrossRef\]](#)
14. Benni, K.E.; Keung, J.; Phannachitta, P.; Monden, A.; Mensah, S. MAHAKIL: Diversity Based Oversampling Approach to Alleviate the Class Imbalance Issue in Software Defect Prediction. *IEEE Trans. Softw. Eng.* **2018**, *44*, 534–550. [\[CrossRef\]](#)
15. Vo, M.T.; Nguyen, T.; Vo, H.A.; Le, T. Noise-Adaptive Synthetic Oversampling Technique. *Appl. Intell.* **2021**, *51*, 7827–7836. [\[CrossRef\]](#)

16. Chawla, N.V.; Bowyer, K.W.; Hall, L.O.; Kegelmeyer, W.P. SMOTE: Synthetic Minority Over-Sampling Technique. *JAIR* **2002**, *16*, 321–357. [\[CrossRef\]](#)
17. He, H.; Bai, Y.; Garcia, E.A.; Li, S. ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning. In Proceedings of the 2008 IEEE International Joint Conference on Neural Networks (IJCNN), Hong Kong, 1–8 June 2008; IEEE: New York, NY, USA, 2008; pp. 1322–1328. [\[CrossRef\]](#)
18. Gong, L.; Jiang, S.; Jiang, L. Tackling Class Imbalance Problem in Software Defect Prediction through Cluster-Based Over-Sampling with Filtering. *IEEE Access* **2019**, *7*, 145725–145737. [\[CrossRef\]](#)
19. Kim, S.; Zhang, H.; Wu, R.; Gong, L. Dealing with Noise in Defect Prediction. In Proceedings of the 2011 33RD International Conference on Software Engineering (ICSE), Honolulu, HI, USA, 21–28 May 2011; IEEE: New York, NY, USA, 2011; pp. 481–490.
20. Alan, O.; Catal, C. Thresholds Based Outlier Detection Approach for Mining Class Outliers: An Empirical Case Study on Software Measurement Datasets. *Expert Syst. Appl.* **2011**, *38*, 3440–3445. [\[CrossRef\]](#)
21. Shi, H.; Ai, J.; Liu, J.; Xu, J. Improving Software Defect Prediction in Noisy Imbalanced Datasets. *Appl. Sci.* **2023**, *13*, 10466. [\[CrossRef\]](#)
22. Puri, A.; Kumar Gupta, M. Improved Hybrid Bag-Boost Ensemble with K-Means-SMOTE-ENN Technique for Handling Noisy Class Imbalanced Data. *Comput. J.* **2020**, *65*, 124–138. [\[CrossRef\]](#)
23. Sáez, J.A.; Luengo, J.; Stefanowski, J.; Herrera, F. SMOTE-IPF: Addressing the Noisy and Borderline Examples Problem in Imbalanced Classification by a Re-Sampling Method with Filtering. *Inf. Sci.* **2015**, *291*, 184–203. [\[CrossRef\]](#)
24. Chen, L.; Fang, B.; Shang, Z.; Tang, Y. Tackling Class Overlap and Imbalance Problems in Software Defect Prediction. *Softw. Qual. J.* **2018**, *26*, 97–125. [\[CrossRef\]](#)
25. Shao, X.; Yan, Y. Noise-Robust Gaussian Distribution Based Imbalanced Oversampling. In *Algorithms and Architectures for Parallel Processing*; Tari, Z., Li, K., Wu, H., Eds.; Springer Nature: Singapore, 2024. [\[CrossRef\]](#)
26. Azhar, N.A.; Pozi, M.S.M.; Din, A.M.; Jatowt, A. An Investigation of SMOTE Based Methods for Imbalanced Datasets with Data Complexity Analysis. *IEEE Trans. Knowl. Data Eng.* **2023**, *35*, 6651–6672. [\[CrossRef\]](#)
27. Luque, A.; Carrasco, A.; Martín, A.; de las Heras, A. The Impact of Class Imbalance in Classification Performance Metrics Based on the Binary Confusion Matrix. *Pattern Recognit.* **2019**, *91*, 216–231. [\[CrossRef\]](#)
28. Vuttipittayamongkol, P.; Elyan, E.; Petrovski, A. On the Class Overlap Problem in Imbalanced Data Classification. *Knowl.-Based Syst.* **2021**, *212*, 106631. [\[CrossRef\]](#)
29. Shen, J.; Li, Z.; Lu, Y.; Pan, M.; Li, X. Mitigating the Impact of Mislabeled Data on Deep Predictive Models: An Empirical Study of Learning with Noise Approaches in Software Engineering Tasks. *Autom. Softw. Eng.* **2024**, *31*, 33. [\[CrossRef\]](#)
30. Ali, A.; Shamsuddin, S.M.; Ralescu, A. Classification with Class Imbalance Problem: A Review. *Int. J. Advance Soft Comput. Appl.* **2015**, *7*, 176–204.
31. Arisholm, E.; Briand, L.C.; Johannessen, E.B. A Systematic and Comprehensive Investigation of Methods to Build and Evaluate Fault Prediction Models. *J. Syst. Softw.* **2010**, *83*, 2–17. [\[CrossRef\]](#)
32. García, V.; Sánchez, J.S.; Marqués, A.I.; Florencia, R.; Rivera, G. Understanding the Apparent Superiority of Over-Sampling through an Analysis of Local Information for Class-Imbalanced Data. *Expert Syst. Appl.* **2020**, *158*, 113026. [\[CrossRef\]](#)
33. Han, H.; Wang, W.-Y.; Mao, B.-H. Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning. In *Advances in Intelligent Computing*; Huang, D.-S., Zhang, X.-P., Huang, G.-B., Eds.; Springer: Berlin/Heidelberg, Germany, 2005; pp. 878–887. [\[CrossRef\]](#)
34. Arafa, A.; El-Fishawy, N.; Badawy, M.; Radad, M. RN-SMOTE: Reduced Noise SMOTE Based on DBSCAN for Enhancing Imbalanced Data Classification. *J. King Saud Univ.—Comput. Inf. Sci.* **2022**, *34 Pt A*, 5059–5074. [\[CrossRef\]](#)
35. Yang, W.; Pan, C.; Zhang, Y. An Oversampling Method for Imbalanced Data Based on Spatial Distribution of Minority Samples SD-KMSMOTE. *Sci. Rep.* **2022**, *12*, 16820. [\[CrossRef\]](#) [\[PubMed\]](#)
36. Gong, L.; Jiang, S.; Wang, R.; Jiang, L. Empirical Evaluation of the Impact of Class Overlap on Software Defect Prediction. In Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), San Diego, CA, USA, 11–15 November 2019; pp. 698–709. [\[CrossRef\]](#)
37. Goel, N.; Singaravelu, M.; Gupta, S.; Namana, S.; Singh, R.; Kumar, R. Parameterized Clustering Cleaning Approach for High-Dimensional Datasets with Class Overlap and Imbalance. *SN Comput. Sci.* **2023**, *4*, 464. [\[CrossRef\]](#)
38. Bernhardt, M.; Castro, D.C.; Tanno, R.; Schwaighofer, A.; Tezcan, K.C.; Monteiro, M.; Bannur, S.; Lungren, M.P.; Nori, A.; Glocker, B.; et al. Active Label Cleaning for Improved Dataset Quality under Resource Constraints. *Nat. Commun.* **2022**, *13*, 1161. [\[CrossRef\]](#) [\[PubMed\]](#)
39. Liu, C.; Jin, S.; Wang, D.; Luo, Z.; Yu, J.; Zhou, B.; Yang, C. Constrained Oversampling: An Oversampling Approach to Reduce Noise Generation in Imbalanced Datasets With Class Overlapping. *IEEE Access* **2022**, *10*, 91452–91465. [\[CrossRef\]](#)
40. Stefanowski, J. Overlapping, Rare Examples and Class Decomposition in Learning Classifiers from Imbalanced Data. In *Emerging Paradigms in Machine Learning*; Ramanna, S., Jain, L.C., Howlett, R.J., Eds.; Springer: Berlin/Heidelberg, Germany, 2013; pp. 277–306. [\[CrossRef\]](#)
41. Tang, W.; Khoshgoftaar, T.M. Noise Identification with the K-Means Algorithm. In Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence, Boca Raton, FL, USA, 15–17 November 2004; pp. 373–378. [\[CrossRef\]](#)
42. Batista, G.E.A.P.A.; Prati, R.C.; Monard, M.C. A Study of the Behavior of Several Methods for Balancing Machine Learning Training Data. *SIGKDD Explor. Newsl.* **2004**, *6*, 20–29. [\[CrossRef\]](#)

43. López, V.; Fernández, A.; García, S.; Palade, V.; Herrera, F. An Insight into Classification with Imbalanced Data: Empirical Results and Current Trends on Using Data Intrinsic Characteristics. *Inf. Sci.* **2013**, *250*, 113–141. [\[CrossRef\]](#)
44. Le, T.; Baik, S.W. A Robust Framework for Self-Care Problem Identification for Children with Disability. *Symmetry* **2019**, *11*, 89. [\[CrossRef\]](#)
45. Le, T.; Lee, M.Y.; Park, J.R.; Baik, S.W. Oversampling Techniques for Bankruptcy Prediction: Novel Features from a Transaction Dataset. *Symmetry* **2018**, *10*, 79. [\[CrossRef\]](#)
46. Lang, C.I.; Sun, F.-K.; Lawler, B.; Dillon, J.; Dujaili, A.A.; Ruth, J.; Cardillo, P.; Alfred, P.; Bowers, A.; Mckiernan, A.; et al. One Class Process Anomaly Detection Using Kernel Density Estimation Methods. *IEEE Trans. Semicond. Manuf.* **2022**, *35*, 457–469. [\[CrossRef\]](#)
47. Jureczko, M.; Madeyski, L. Towards Identifying Software Project Clusters with Regard to Defect Prediction. In Proceedings of the 6th International Conference on Predictive Models in Software Engineering, Timișoara, Romania, 12–13 September 2010; PROMISE '10; Association for Computing Machinery: New York, NY, USA, 2010; pp. 1–10. [\[CrossRef\]](#)
48. D'Ambros, M.; Lanza, M.; Robbes, R. An Extensive Comparison of Bug Prediction Approaches. In Proceedings of the 2010 7th IEEE Working Conference on Mining Software Repositories (MSR 2010), Cape Town, South Africa, 2–3 May 2010; pp. 31–41. [\[CrossRef\]](#)
49. Shepperd, M.; Song, Q.; Sun, Z.; Mair, C. Data Quality: Some Comments on the NASA Software Defect Datasets. *IEEE Trans. Softw. Eng.* **2013**, *39*, 1208–1215. [\[CrossRef\]](#)
50. Wu, R.; Zhang, H.; Kim, S.; Cheung, S.-C. ReLink: Recovering Links between Bugs and Changes. In Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering, Szeged, Hungary, 5–9 September 2011; ACM: Szeged, Hungary, 2011; pp. 15–25. [\[CrossRef\]](#)
51. D'Ambros, M.; Lanza, M.; Robbes, R. Evaluating Defect Prediction Approaches: A Benchmark and an Extensive Comparison. *Empir. Software Eng.* **2012**, *17*, 531–577. [\[CrossRef\]](#)
52. New Applications of Ensembles of Classifiers | Pattern Analysis and Applications. Available online: <https://link.springer.com/article/10.1007/s10044-003-0192-z> (accessed on 21 September 2024).
53. Fan, G.; Diao, X.; Yu, H.; Yang, K.; Chen, L. *Software Defect Prediction via Attention-Based Recurrent Neural Network*; Wiley: Hoboken, NJ, USA, 2019. [\[CrossRef\]](#)
54. Mahmood, Z.; Bowes, D.; Lane, P.C.R.; Hall, T. What Is the Impact of Imbalance on Software Defect Prediction Performance? In Proceedings of the 11th International Conference on Predictive Models and Data Analytics in Software Engineering, Beijing, China, 21 October 2015; PROMISE '15. Association for Computing Machinery: New York, NY, USA, 2015; pp. 1–4. [\[CrossRef\]](#)
55. Macbeth, G.; Razumiejczyk, E.; Ledesma, R.D. *Cliff's Delta Calculator: A Non-Parametric Effect Size Program for Two Groups of Observations*; ResearchGate: Berlin, Germany, 2010. [\[CrossRef\]](#)

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.